# Machine Learning Assignment Report

## Two-Player Sushi Go Reinforcement Learning System

Chenhe Yuan, Student Number: 22003915 Guanfang Lai, Student Number: *fill in*
Simon Zhu, Student Number: 22020163 Xander Mills, Student Number: *fill in*

March 27, 2026

**Abstract**

This report presents a deterministic two-player Sushi Go reinforcement learning system implemented in Python. The project combines a rule-correct multi-round game environment, baseline agents, MaskablePPO training, statistical evaluation, and an auxiliary LLM-based explain and coaching layer. The core technical challenge is a sparse-reward sequential decision problem with state-dependent legal actions caused by the chopsticks mechanic and delayed end-of-game pudding scoring. To address that setting, the system uses a fixed-length observation vector, an explicit legal-action mask, and MaskablePPO for policy optimisation. The report documents the current repository state and adds a full reproduced training result rather than relying on older named checkpoints. A fresh 1,000,000-step curriculum run from the current scripts produces the tracked model `repro_curriculum_1m`, which achieves a deterministic 96.6% win rate against a random opponent and 95.7% against the heuristic baseline over 1000 episodes.

## 1 Introduction

This project develops a research-oriented reinforcement learning benchmark around a deterministic two-player Sushi Go environment. The design goals are threefold: correctness of rules and scoring, reproducibility of trajectories and results, and trainability under sparse terminal rewards. The system therefore separates pure scoring logic from environment transitions, exposes a Gymnasium-style API for training, and includes baseline opponents to support controlled evaluation.

The implemented ruleset is the repository's current three-round variant rather than a reduced one-round prototype. Each round deals ten cards per player from a fixed 108-card deck. Both players act simultaneously, the chosen cards are resolved in play order, hands are swapped, and scoring is applied at round end. Pudding is accumulated across rounds and scored only once at the end of the game. These mechanics create a medium-horizon strategic task in which local drafting decisions interact with round-level maki races and game-level pudding incentives.

The report has two purposes. First, it documents the current repository in a way that is faithful to the code. Second, it turns the project from an implementation description into a finished experimental report by adding quantitative evidence: baseline comparisons, reproducibility checks, and a fresh end-to-end training run using the current scripts and environment.

# 2 Game Rules and Environment

## 2.1 Game flow

The game is played by exactly two players over three rounds. At the start of each round, the environment shuffles the fixed deck with a seeded NumPy random generator and deals ten cards to each player. On every turn, both players choose actions simultaneously. The chosen cards are added to the current round pile in play order, the remaining hands are exchanged, and the turn counter advances. After ten turns, the round is scored, round piles are cleared, and the next round begins. After the third round, end-of-game pudding scoring is applied and the episode terminates.

## 2.2 Scoring logic

The implemented card set contains Tempura, Sashimi, Dumpling, Maki-1/2/3, Nigiri-1/2/3, Wasabi, Pudding, and Chopsticks. Scoring follows the rules encoded in `docs/RULES.md` and `rules.py`:

- Tempura scores 5 points per pair.
- Sashimi scores 10 points per triple.
- Dumpling uses the capped progression $0, 1, 3, 6, 10, 15$.
- Wasabi triples only the next later Nigiri in play order.
- Maki is scored comparatively within a round; in two-player ties for first, the first-place bonus is split as 3 points each.
- Pudding is accumulated across all three rounds and scored only once at game end; the implementation uses `penalty_for_last=False`, so the player with fewer puddings is not penalised.

## 2.3 Reward design

The project uses a sparse terminal reward. All intermediate rewards are zero. After the third round, the agent receives

$$r_{\text{terminal}} = \texttt{my_score} - \texttt{opp_score}.$$

This design keeps optimisation aligned with the actual objective of the game and avoids reward-shaping artefacts that could bias the agent toward locally attractive but strategically inferior behaviour.

# 3 System Design and Technical Stack

## 3.1 Architecture

The implementation is modular:

- `rules.py`: pure deterministic card definitions, deck composition, and scoring helpers.
- `env.py`: Gymnasium-style environment with multi-round game state, action masking, simultaneous resolution, and seeded determinism.
- `agents/random_agent.py` and `agents/heuristic_agent.py`: baseline opponents.

- `train.py`: MaskablePPO training with random, heuristic, mixed, or curriculum opponents.
- `eval.py`: statistical evaluation and reproducibility checks.
- `llm_assistant.py`, `llm_demo.py`, `llm_eval.py`: optional explain/coach tooling around a frozen policy.

This separation is important methodologically. Pure scoring functions can be unit-tested independently, while environment transitions and episode mechanics can be integration-tested without entangling them with policy training code.

### 3.2 Technical stack

| Component | Purpose in the project |
|---|---|
| Gymnasium | Standard environment interface for `reset`/`step` transitions and policy integration. |
| NumPy | Deterministic numerical operations, state encoding, and seeded stochastic control. |
| Stable-Baselines3 + SB3-Contrib | PPO training with explicit legal-action masking via MaskablePPO. |
| PyTest | Unit and integration testing for rules, environment behaviour, and LLM helper code. |
| Matplotlib | Optional plotting utilities for checkpoint and learning-curve evaluation. |

Table 1: Technical stack and rationale.

### 3.3 Observation space

The environment exposes a fixed-size observation vector `Box(shape=(176,), dtype=float32)`. The vector is deliberately engineered rather than learned from raw card sequences, because the game state is compact and most strategically relevant information can be represented explicitly.

| Feature section | Content | Dimension |
|---|---|---|
| My played counts | Count of each card type in the agent's current round pile | 12 |
| Opponent played counts | Count of each card type in the opponent's current round pile | 12 |
| My hand counts | Count of each card type in the current hand | 12 |
| Hand slot encoding | One-hot encoding of each of the 10 hand slots over 13 slot types (12 cards + empty) | 130 |
| Scalar features | Turn, hand size, unpaired wasabi, maki counts, round index, pudding counts, cumulative scores | 10 |
| Total | | 176 |

Table 2: Observation encoding used by the policy.

The slot encoding is necessary because actions are indexed by hand position rather than by card identity alone. The additional round, pudding, and cumulative score features allow the policy to condition its drafting behaviour on cross-round game state rather than only the local round context.

### 3.4 Action space and masking

The action space is `Discrete(100)`. Actions 0 through 9 correspond to playing a single card from the current hand index. Actions 10 through 99 encode ordered two-card chopsticks plays. Legal actions vary by state: single-card plays are legal only for currently occupied hand slots, and chopsticks actions are legal only when the player already has a chopsticks card on the table and at least two cards in hand.

This state-dependent legality is one of the main reasons MaskablePPO is appropriate for the task. Instead of learning to avoid invalid actions through penalties, the algorithm samples only from the explicit legal-action mask, which eliminates an unnecessary source of instability.

### 3.5 Reproducibility

Reproducibility is implemented directly in the environment design. All stochasticity is sourced from NumPy's `default_rng`. Calling `reset(seed=...)` recreates the random generator with the specified seed, and the optional `fixed_episode_seed` configuration forces repeated episodes to use identical deck shuffles and opponent sampling. The repository also includes a reproducibility sanity check in `eval.py`; under the current codebase, that check passes for seed 17 and confirms that repeated runs with the same action sequence produce identical trajectories and terminal rewards. The new headline model reported here was trained directly from the version-controlled scripts, which makes the result materially easier to reproduce than relying on an older artifact with incomplete provenance.

## 4 Model Choice and Auxiliary LLM Layer

The core learning problem has three properties that shape model choice: sparse terminal reward, dynamic legal actions, and delayed strategic effects across multiple rounds. Chopsticks create combinational action legality, while pudding and wasabi create delayed value that is not captured by immediate scoring alone. This makes naive value-based approaches with penalty-style invalid-action handling less attractive.

For that reason, the project uses MaskablePPO with an MLP policy over the 176-dimensional state representation. PPO's clipped updates are a pragmatic choice for medium-horizon sparse-reward optimisation, while the legal-action mask solves the invalid-action problem cleanly at sampling time.

The repository also contains an LLM explain/coach layer. This component is not part of the RL optimisation loop and is therefore outside the core performance evaluation. Instead, it wraps a frozen policy and uses either OpenAI, Gemini, or deterministic fallback templates to turn the policy's recommendations into human-readable move explanations. In methodological terms, this layer improves interpretability and interactive usability, but it does not affect the underlying gameplay results reported in this document.

# 5 Experimental Methodology

## 5.1 What was evaluated

The main empirical goal of this report is to show that the current training scripts can reproduce a strong policy from scratch. Instead of selecting among older stored artifacts, the experiments therefore focus on one fresh run produced directly from the repository:

- `runs/repro_curriculum_1m.zip`
- `runs/repro_curriculum_1m.vecnormalize.pkl`

This checkpoint was trained with curriculum opponents for 1,000,000 timesteps and evaluated under the same deterministic protocol as the baselines.

## 5.2 Protocol

Three types of experiments were run:

1. Baseline controls over 1000 episodes:
   - random vs random
   - heuristic vs random
2. Reproducibility sanity check:
   - fixed-seed trajectory replay with identical action sequence
3. Fresh model training and evaluation:
   - one 1,000,000-step curriculum MaskablePPO run
   - deterministic policy evaluation over 1000 episodes against both random and heuristic opponents

The evaluation metric suite includes wins, losses, ties, win rate including ties, mean score difference, and score-difference dispersion. For the headline 1000-episode win rates, approximate 95% normal confidence intervals are reported.

## 5.3 Verification and software quality

The full repository test suite passes on the current codebase: 51 tests passed. These tests cover scoring, environment determinism, action legality, observation encoding, and LLM helper behaviour. That matters because experimental results are only meaningful if the environment itself is trusted.

# 6 Results

## 6.1 Baseline controls

These controls validate that the evaluation pipeline is behaving sensibly. Random vs random is near-symmetric, while the heuristic baseline is substantially stronger than random, making it a meaningful opponent for RL checkpoint evaluation.

| Matchup | Episodes | Win rate | Ties | Mean score diff | Std. dev. |
|---|---|---|---|---|---|
| Random vs random | 1000 | 0.476 | 0.049 | +0.065 | 10.669 |
| Heuristic vs random | 1000 | 0.760 | 0.023 | +7.911 | 10.518 |

Table 3: Baseline control experiments. Random vs random is effectively balanced, while the heuristic clearly outperforms random play.

| Setting | Value |
|---|---|
| Algorithm | MaskablePPO |
| Timesteps | 1,000,000 |
| Seed | 0 |
| Opponent mode | curriculum |
| Curriculum stages | `0.0:200000,0.2:200000,0.5:300000,0.8:300000` |
| Observation normalisation | `VecNormalize` enabled |
| Saved model | `runs/repro_curriculum_1m.zip` |
| Saved statistics | `runs/repro_curriculum_1m.vecnormalize.pkl` |

Table 4: Configuration of the fresh reproduced training run reported in this document.

### 6.2 Training configuration

### 6.3 Fresh 1M model evaluation

| Opponent | Episodes | Wins | Losses | Ties | Win rate | Mean score diff |
|---|---|---|---|---|---|---|
| Random | 1000 | 966 | 23 | 11 | 0.966 | +17.573 |
| Heuristic | 1000 | 957 | 35 | 8 | 0.957 | +16.238 |

Table 5: 1000-episode deterministic evaluation for the fresh reproduced checkpoint `repro_curriculum_1m`.

Approximate 95% confidence intervals for win rate including ties are:

- vs random: $0.966\ [0.955, 0.977]$
- vs heuristic: $0.957\ [0.944, 0.970]$

These numbers are well above the heuristic baseline's 0.760 win rate against random play, which indicates that the reproduced RL policy has learned strategic behaviour that materially exceeds the hand-coded rule baseline.

## 7 Discussion

### 7.1 What the experiments show

The experiments support three main conclusions.

First, the environment is learnable. The reproduced model comfortably outperforms both random play and the heuristic baseline, which shows that the reward, state encoding, and action masking are sufficient for strong policy learning.

Second, opponent diversity matters. The curriculum schedule gradually increases exposure to the heuristic opponent, and the resulting model performs strongly against both evaluation opponents. That supports the design choice of mixed-opponent training rather than training only against random play.

Third, the project is now in a better reproducibility state than before. The reported best model is no longer just an inherited artifact with unclear provenance; it is a checkpoint trained directly from the current scripts with preserved command lines and matching evaluation notes.

### 7.2 Limitations

The report also exposes several limitations of the current project state.

- Full training provenance is incomplete for some stored checkpoints. The repository contains artifacts, but not a perfectly preserved training log for every model.
- The reproduced model is strong under the chosen curriculum and seed, but this report does not quantify variance across many independent training seeds.
- Evaluation in this report focuses on self-play against scripted baselines, not against human players.
- The game is still a constrained Sushi Go variant: only two players, exactly three rounds, and a fixed card set.
- The LLM explain/coach component improves usability and interpretability, but it has not been evaluated here for human usefulness beyond repository tooling support.

### 7.3 What should be improved next

The most useful next steps are straightforward:

- preserve training metadata automatically for every checkpoint,
- run multiple 1M-step seeds and report variance,
- add plots of performance versus training steps,
- and evaluate whether the LLM coach actually helps human decision making in interactive play.

## 8 Conclusion

This project successfully implements a deterministic, rule-correct, and testable Sushi Go reinforcement learning system. The environment design is technically coherent: scoring is isolated in pure functions, legal-action masking is explicit, and seeded determinism is built into the simulation. The baseline controls behave as expected, the reproducibility check passes, and the full test suite succeeds.

Most importantly, the report now demonstrates empirical performance rather than only describing an architecture. A fresh 1,000,000-step curriculum training run from the current repository produces `repro_curriculum_1m`, which achieves 96.6% win rate against random play and 95.7% against the heuristic baseline over 1000 deterministic episodes. That result supports the central claim of the project: a masked PPO policy can learn a strong strategy in this constrained, sparse-reward Sushi Go environment, and the current codebase can reproduce that result directly.

# A Reproducibility Commands

The following commands were used to produce the main numbers reported above from the repository root:

```
conda run -n sushigo-rl env PYTHONPATH=src python -m pytest -q

conda run -n sushigo-rl env PYTHONPATH=src MPLCONFIGDIR=/tmp/matplotlib \
python -m sushigo_rl.eval --opponents none --baselines random_vs_random \
  --episodes 1000

conda run -n sushigo-rl env PYTHONPATH=src MPLCONFIGDIR=/tmp/matplotlib \
python -m sushigo_rl.eval --opponents none --baselines heuristic_vs_random \
  --episodes 1000

conda run -n sushigo-rl env PYTHONPATH=src MPLCONFIGDIR=/tmp/matplotlib \
python -m sushigo_rl.eval --opponents none --baselines none --repro-check \
  --repro-seed 17

PYTHONPATH=src /home/simon/miniconda3/envs/sushigo-rl/bin/python \
  -m sushigo_rl.train --opponent-mode curriculum \
  --curriculum-stages "0.0:200000,0.2:200000,0.5:300000,0.8:300000" \
  --timesteps 1000000 --seed 0 --vecnorm \
  --vecnorm-path runs/repro_curriculum_1m.vecnormalize.pkl \
  --model-out runs/repro_curriculum_1m --checkpoint-every 100000 \
  --checkpoint-dir runs/repro_curriculum_1m_checkpoints \
  --checkpoint-prefix model --verbose 0

conda run -n sushigo-rl env PYTHONPATH=src MPLCONFIGDIR=/tmp/matplotlib \
python -m sushigo_rl.eval --model runs/repro_curriculum_1m.zip \
  --vecnorm-path runs/repro_curriculum_1m.vecnormalize.pkl \
  --episodes 1000 --opponents both --baselines none --deterministic
```